Universidade da Beira Interior
Faculdade de Engenharia
Departamento de Informática

© Pedro R. M. Inácio (inacio@di.ubi.pt), 2025/26

Propostas para Trabalhos de Grupo
Team Work Proposals

Segurança Informática & Cibersegurança
Computer Security & Cybersecurity

Departamento de Informática
Department of Computer Science
Universidade da Beira Interior
University of Beira Interior

Pedro R. M. Inácio
inacio@di.ubi.pt
2025/26

Copyright © 2025 All rights reserved.

Conteúdo

1 Introdução **3**

1.1 Entrega do Trabalho 4

1.2 Sugestão de Alinhamento para a Apresentação 4

2 Propostas de Trabalho **4**

2.1 SAFE-DEPOSIT-BOX: Um Sítio Seguro para Guardar Mensagens 4

2.2 OK-Eu-CONFESSO: Recibo de Leitura com Garantia Criptográfica 6

2.3 ASSINONDEQUISERES: Assinaturas Digitais para Documentos com Alternativas 8

2.4 EU-PAGOTAMANHÃ: Contratos e Empréstimos Seguros 9

2.5 SEE-U-L4TER: Uma Cápsula do Tempo Criptográfica 11

2.6 CANTTOUCHME: Um Livro de Registos Pessoal e Seguro 12

2.7 NOTHING2LOSE: Uma Lotaria que Compromete Tempo e Prémios 13

2.8 CHALLENGE-ACCEPTED: Um Sistema de Publicação de Desafios Criptográficos 14

2.9 MON-AMOUR: Um Sistema para Trocar Cartas de Amor, mas com um Senão... 15

2.10 CTRL+C: Um Gestor de *Clipboard* Porreirinho 17

2.11 TWO-HEADED-BEAST: Aplicação para Transmitir Ficheiros entre Dois Utilizadores 18

2.12 iCHATO: Aplicação para Conversação Online com Mensagens Assinadas . . 19

2.13 MR.SMITH: Plataforma para Cifrar e Assinar usando um Agente de Confiança 20

Copyright © 2026 All rights reserved.

1 Introdução

Introduction

As seguintes propostas de trabalho foram elaboradas no contexto da unidade curricular de Segurança Informática e servem primariamente o propósito de *exemplificar* o funcionamento de primitivas da criptografia e de aplicações que as integrem. Como tal, estas propostas não replicam necessariamente, ou em todo o detalhe, aplicações ou tecnologias atualmente em uso.

The following proposals were elaborated within the context of the subject of Computer Security and they serve primarily the purpose of exemplifying the functioning of criptography related primitives and of applications that make use of them. As such, they may not all replicate real life applications or technologies used nowadays.

Notem também que os estudantes são livres de submeter uma nova proposta, desde que esteja em concordância com os objetivos da unidade curricular. Adicionalmente, a proposta tem de ser discutida com o Professor e descrita num formato semelhante ao das demais antes de ser aceite como um tópico de trabalho válido.

Notice also that the students are free to submit a new proposal, as long as it is in accordance with the objectives of this subject. Additionally, the proposal has to be discussed with the Professor and described in a format similar to the one included herein prior to being accepted as a valid work topic.

Como de resto discutido durante as aulas, as equipas devem ter entre cinco e seis elementos, e a implementação destas propostas deve ser complementada com artefactos que provem a autoria e originalidade (e.g., mostrando evidências do uso de ferramentas de gestão do projeto de software) e com uma apresentação em que se discutem as dificuldades encontradas ao longo do trabalho e se justificam todas as escolhas tomadas. A apresentação ocorre no final do semestre, recorrendo a um pequeno conjunto de diapositivos.

As discussed in the classes, the teams should be formed by five or six students, and the implementation of these proposals should be complemented with artifacts that prove the authoring and originality (e.g., showing evidences of the usage of software management tools) and with a presentation in which the difficulties faced along the work are discussed and the choices that have been taken are justified. The presentation will take place at the end of the semester, resorting to a small set of slides.

As aplicações podem ser implementadas recorrendo a qualquer estúdio ou ambiente de desenvolvimento integrado, sendo esse pormenor deixado ao critério dos estudantes. São também livres de decidir a linguagem de programação ou as tecnologias *web* (se aplicáveis) a usar (a não ser que os meios a utilizar estejam diretamente indicados na proposta). Contudo, por favor tomem em consideração os dois apontamentos que se seguem: (i) não presumam nem estejam à espera que o Professor seja capaz de responder a todas as questões específicas às tecnologias que escolheu (por exemplo, não espere que ele saiba como trabalhar simultaneamente com o Netbeans, Eclipse, Visual Studio, etc.) e, (ii) a aplicação deve estar a funcionar corretamente no fim do semestre, independentemente das ferramentas ou tecnologias utilizadas. Alguns dos problemas que encontrarem durante o desenvolvimento destes projetos podem já ter sido tratados ou resolvidos nas aulas, e são livres de reutilizar, se aplicável, qualquer pedaço de código ou recurso desenvolvido ao longo das mesmas.

The applications can be implemented resorting to any development studio or integrated development environment of your choice. Students are free to decide the programming language or the web technologies (if applicable) they are going to use also (unless the means that should be used are directly indicated in the work proposal). Nonetheless, please take the following two remarks into consideration: (i) do not expect the Professor to be able to answer all technology specific questions (e.g., do not expect him to know how to work simultaneously with Netbeans, Eclipse, Visual Studio, etc.) and, (ii) the application should be correctly functioning at the end of the semester, independently of those choices. Some of the problems you may face during the development of these projects may have been already addressed during the practical classes, and you are free to reuse, if applicable, any code developed along the classes.

É da responsabilidade dos estudantes a pesquisa de detalhes específicos à solução dos

Copyright © 2026 All rights reserved.

problemas propostos, assim como de maneiras de testar a validade dessas soluções. Para além de referirem a solução do problema na apresentação, os estudantes deve também descrever o modo como validaram o que foi feito (se aplicável) e incluir alguma teoria de como foi conseguido.
It is of the responsibility of the students to search for specific details concerning the proposal at hands and for ways to validate the solutions. Besides including the solution to the problem in the presentation, students should also describe the means used to validate the work (if applicable) and include some of the theory behind what was done.

1.1 Entrega do Trabalho
Delivery of the Works

A entrega do trabalho é feita única e exclusivamente via *moodle* até às 23:59 do dia de entrega do trabalho e os **nomes dos ficheiros devem seguir a especificação** incluída na secção dedicada a essa entrega na área da unidade curricular naquela plataforma. Por cada dia de atraso na entrega de qualquer elemento do trabalho (código de implementação, *scripts* de instalação ou outros artefactos), descontam-se 0,5 valores (aos 5).

1.2 Sugestão de Alinhamento para a Apresentação
Suggested Lineup for the Presentation

A defesa do trabalho deve ser acompanhada por um breve conjunto de diapositivos. A apresentação oral pelos elementos do grupo não pode demorar mais do que 10 minutos. Devem considerar fazer um conjunto de 10 diapositivos para guiar o discurso com o seguinte alinhamento (façam as adaptações que considerem necessárias):

- 1 diapositivo com o titulo do trabalho e elementos do grupo;
- 1 diapositivo com os objetivos do trabalho;
- 1 diapositivo dedicado ao levantamento de requisitos de segurança;
- 1 diapositivo dedicado à Engenharia do Software;
- 1 diapositivo dedicado à implementação;
- 1 diapositivo dedicado à apresentação do sistema/aplicação desenvolvida (este diapositivo é só para lembrar para fazer o *switch* para um demonstrador real ou para um video da aplicação a correr);
- 1 diapositivo a referir os testes efetuados;
- 1 diapositivo com a análise crítica;
- 1 diapositivo dedicado aos objetivos alcançados / conclusões e trabalho futuro.
- 1 diapositivo a dizer `Bem haja pela atenção. Perguntas?`

2 Propostas de Trabalho
Work Proposals

2.1 SAFE-DEPOSIT-BOX: Um Sítio Seguro para Guardar Mensagens
SAFE-DEPOSIT-BOX: A Safe Place to Store Messages

O objetivo principal deste trabalho é engendrar e desenvolver um sistema que imite (quanto baste), no mundo digital, o conjunto de cofres seguros que os bancos por vezes disponibilizam, mas apenas para mensagens. Assim, será preciso implementar um sistema (e.g.,

Copyright © 2026 All rights reserved.

uma aplicação web) que permita que um(a) utilizador(a) escreva uma mensagem e que esta seja guardada cifrada, sendo em troca oferecido ao utilizador uma forma da decifrar quando for preciso. Repare-se que, no mundo real, é possível que uma pessoa possa dar acesso a um dos seus cofres a outra pessoa. Assim, o sistema também deve permitir essa possibilidade, de várias formas, como descrito em baixo.

Para ter uma ideia de como o sistema pode funcionar, considere o seguinte cenário de utilização simplificado:

- um utilizador acede ao sistema (web) e insere uma mensagem numa caixa de texto de uma página web;
- o sistema gera uma (ou várias - ver em baixo) chave(s) de cifra e cifra o texto com uma cifra segura, guardando-o numa base de dados cifrado;
- o sistema mostra a chave de cifra ao utilizador e pede-lhe para que este/a a guarde de forma segura;
- mais tarde, o/a mesmo/a utilizador/a acede ao sistema para obter a mensagem (numa segunda página web), e insere a chave de cifra;
- a partir da chave de cifra (e de algum metadado adicional), o sistema identifica o cofre e decifra a mensagem, mostrando-a ao utilizador.

A aplicação a desenvolver só terá um ator (o `utilizador` normal), e o conjunto de funcionalidades básicas a suportar pela aplicação (para além da que vaticina o cenário acima) são:

1. a aplicação permite que um utilizador deposite uma mensagem num cofre (conforme cenário em cima), ficando esta cifrada com AES-256-CBC;
2. a aplicação permite guardar a mensagem usando uma palavra-passe escolhida pelo utilizador, ao invés de lhe ser atribuida uma chave de cifra simétrica. Nesse caso, a chave de cifra deve ser uma derivação da palavra-passe usando um algoritmo seguro;
3. a aplicação permite guardar a mensagem usando criptografia de chave pública RSA ou ElGamal, ao invés de lhe ser atribuida uma chave de cifra simétrica. Nesse caso, deve ser gerado um par de chaves para o utilizador aquando do depósito, e fornecida a respetiva chave privada ao utilizador;
4. a aplicação deve permitir que um utilizador abra o cofre (i.e., decifre a mensagem) mais tarde com qualquer um dos métodos acima descritos (i.e., fornecendo a chave de cifra simétrica, a palavra-passe ou a chave privada);
5. a aplicação deve guardar um código de autenticação de mensagens de todos os criptogramas (i.e., cofres fechados) usando HMAC-SHA256, que constrói usando uma chave de integridade fixa do próprio sistema;
6. sempre que alguém tenta a abertura de um cofre (i.e., a decifra de um criptograma) o sistema verifica primeiro o respetivo código de autenticação de mensagens e mostra o resultado no ecrã, dizendo se o criptograma está correto ou não;
7. a aplicação deve guardar uma assinatura digital de todos os textos-limpos (i.e., antes de uma mensagem ser cifrada).

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. a aplicação deve permitir escolher a cifra simétrica que é utilizada entre AES-256-CBC, AES-256-CTR e ChaCha20;

Copyright © 2026 All rights reserved.

2. a apolicação deve verificar a assinatura digital depois de um cofre ser aberto, apresentando o resultado dessa verificação ao utilizador;
3. a aplicação permite escolher a função de *hash* usada para produzir os códigos de autenticação de mensagens, entre SHA256 ou SHA512;
4. a aplicação permite escolher a função de *hash* usada para produzir as assinaturas, e outros parâmetros relacionados com a segurança (e.g., o tamanho do módulo das chaves RSA);
5. outras funcionalidades que considere interessantes.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.2 OK-Eu-CONFESSO: Recibo de Leitura com Garantia Criptográfica

HERE-I-CONFESS: Read Receipt with Cryptographic Assurance

Em algumas situações existe a necessidade de enviar e-mails com pedido de notificação de entrega e de leitura. Nesses casos, os sistemas e aplicações de e-mail enviam de volta uma mensagem protocolada quando uma mensagem é rececionada na caixa de chegada de um destinatário, ou quando este(a) a abre na aplicação, respetivamente. Contudo, esta última não significa exatamente que a pessoa leu a mensagem. Estes artefactos são todos automáticos.

O objetivo deste trabalho é construir uma artimanha digital que leve o(a) utilizador(a) a confirmar a receção e a garantir que lê a mensagem, pedindo-lhe que confirme explicitamente através de duas ações. Para isso, é necessário desenvolver um sistema (web) que envie e-mails para um(a) qualquer destinatário(a), mas que os cifre parcialmente, sendo que a chave para os decifrar só pode ser obtida numa página do sistema através da apresentação de um código, de uma chave privada, e de uma declaração em conforme o destinatário vai ler o e-mail. Para obter uma ideia do que é pretendido, considere os dois cenários de utilização hipotéticos:

– Cenário 1 –

- Um utilizador acede ao sistema (web) e regista-se pela primeira vez, recebendo uma palavra-passe de 16 caracteres. O sistema é que gera a palavra-passe e a entrada no sistema não precisa de nome de utilziador (a palavra-passe grande chega);
- Aquando do registo, o sistema gera um par de chaves RSA, que o utilizador guarda no seu computador ou no próprio sistema;
- Dentro do sistema, o utilizador pode enviar um e-mail para alguém, escrevendo o endereço destino, o assunto e o corpo do e-mail;
- O sistema, depois do passo anterior, cifra a mensagem no corpo do e-mail com uma chave simétrica gerada pseudo-aleatoriamente a partir de um código, calcula um código de autenticação de mensagens e envia a mensagem para o e-mail do(a) destinatário(a). A mensagem está cifrada, mas tem um texto no início semelhante a – `Se quiser ler este e-mail, aceda ao sistema https://OK-Eu-CONFESSO.xxx e confirme a receção com o código XXXXXXXXXXXX. Ser-lhe-á decifrada e mostrada a mensagem.`

– Cenário 2 –

- Um utilizador recebe um e-mail com o corpo do e-mail cifrado e com a indicação para se dirigir ao site, e clica na hiperligação;

Copyright © 2026 All rights reserved.

- Uma vez na página, são-lhe pedidos (i) o corpo da mensagem, (ii) o código e (iii) a sua chave privada (se o utilizador não tiver chave privada, é-lhe pedido um registo);
- Após fornecer os três elementos anteriores, o sistema apresenta duas caixas de diálogo, uma a perguntar se acusa receção do e-mail (`Sim` ou `Não`) e outra a perguntar se confirma que irá ler a mensagem (`Sim` ou `Não`);
- Se o utilizador responder a ambas as questões afirmativamente, o sistema decifra a mensagem e mostra-a ao utilizador;
- Simultaneamente, se o utilizador responder a ambas as questões afirmativamente, o sistema guarda a informação de que o(a) destinatário recebeu e leu a mensagem. Adicionalmente, o sistema gera um recibo de receção e de leitura assinados digitalmente pelo(a) destinatário(a).

A aplicação a desenvolver só terá um ator (o `utilizador` normal), mas que se pode comportar como emissor ou como recetor, e o conjunto de funcionalidades básicas a suportar pela aplicação (para além das que vaticinam os cenários acima) são:

1. a aplicação permite o registo simples de um utilizador. O registo gera uma palavra-passe de 16 caracteres e não precisa de nome de utilizador;
2. durante o registo (mencionado no ponto anterior), o sistem gera também um par de chaves RSA, cuja chave privada é cifrada com uma derivação segura da palavra-passe de acesso. A cifra a utilizar é a AES-256-CBC. As duas chaves (pública e privada) são dadas ao utilizador aquando do seu registo;
3. quando o utilizador usa o sistema para enviar uma mensagem, este pede-lhe a palavra-passe de acesso, o e-mail destino, o assunto e o corpo do e-mail. O sistema gera um código aleatório de 32 caracteres hexadecimal e deriva uma chave de cifra de 256 bits usando uma função segura de derivação de chaves de cifra (e.g., PBKDF2);
4. o sistema cifra o corpo do e-mail com aquela chave de cifra de 256 bits usando AES-256-CTR, coloca o código no corpo do e-mail (ver cenário) e envia a mensagem;
5. o sistema guarda uma representação segura da palavra-passe dos utilizadores, e não a própria palavra-passe;
6. quando o utilizador usa o sistema para ver uma mensagem, este pede-lhe o código de acesso, a sua palavra-passe de utilizador, a sua chave privada e o corpo do e-mail. Após correr o *Cenário 2* em cima, o sistema decifra a mensagem e gera uma assinatura SHA256withRSA do *recibo de leitura*;
7. um emissor pode, mais tarde, ir verificar se determinada mensagem tem *recibo de leitura*. Caso tenha, o sistema deve verificar e mostrar a validade da assinatura desse *recibo*.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. o sistema integra códigos de autenticação de mensagens HMAC-SHA256 para verificar a integridade das mensagens. Nesse caso, o sistema não só constroi e envia um código MAC, como o pede aquando o destinatário tenta decifrar a mensagem;
2. o sistema envia notificações aos emissores (por e-mail) quando determinada mensagem tem um *recibo de leitura;*
3. a aplicação deve permitir escolher a cifra que é utilizada entre AES-256-CBC e AES-256-CTR para cifra do ficheiro da chave privada;

Copyright © 2026 All rights reserved.

4. a aplicação permite escolher a função de *hash* usada para produzir as assinaturas, e outros parâmetros relacionados com a segurança (e.g., o tamanho do módulo das chaves RSA);

5. outras funcionalidades que considere interessantes.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.3 ASSINONDEQUISERES: Assinaturas Digitais para Documentos com Alternativas

ASSINONDEQUISERES: Digital Signatures for Documents with Alternatives

O ato de assinatura presume que o conteúdo do documento não irá mudar mais após a assinatura, e a assinatura digital, enquanto mecanismo criptopgráfico, garante precisamente a autenticidade e integridade absoluta da totalidade do conteúdo assinado. Este caso de uso principal satisfaz a esmagadora maioria dos cenários de aplicação. Contudo, podem existir casos em que é necessário assinar um documento que tem alguns campos que ficam em aberto (i.e., podem mudar depois do documento ser assinado) ou que podem tomar um valor entre vários possíveis (sendo que o assinante concorda com qualquer um desses valore, mas não com todos simultaneamente). Por exemplo, imagine que estava a assinar um documento em que a data não precisa ficar fixa (i.e., assina o conteúdo do ficheiro, mas a data pode ser o dia de hoje ou de amanhã); ou imagine que está a assinar um documento em que concorda comprar uma camisa com a cor vermelha, verde ou azul, mas que deixa a escolha final dessa cor para o departamento de compras.

O objetivo deste trabalho é pensar e desenvolver uma aplicação que permita assinar documentos com caracteristicas especiais referidas em cima. A aplicação deve ser o mais simples de utilizar possível e assumir que terá de tratar de tudo em prol do utilizador, inclusive a geração das chaves. Para obter uma ideia do que é pretendido, considere o seguinte cenário de utilização hipotético:

- Um utilizador entra na aplicação e é-lhe perguntado se já tem um par de chaves para assinatura ou quer gerar um par de chaves;
- Esse utilizador escolhe, por exemplo, que quer gerar um par de chaves, e a aplicação faz-lhe isso, guardando a chave privada num ficheiro cifrado;
- De seguida, a aplicação pede ao utilizador que escreve o texto (do documento) a assinar, indicando que pode deixar palavras como `[PLACEHOLDER1]` ou `[PLACEHOLDER2]` em locais onde o texto não está definido (ver exemplos em cima);
- A aplicação pede então que defina o que quer colocar nos `[PLACEHOLDERs]` (e.g., se o texto é livre e não vai ser assinado; ou se é um conjunto de escolhas múltiplas);
- Finalmente, a aplicação elabora uma ou mais assinaturas do documento e guarda os respetivos ficheiros.

A aplicação a desenvolver só terá um ator (o `utilizador` normal) e o conjunto de funcionalidades básicas a suportar pela aplicação são:

1. a aplicação pergunta ao utilizador se quer gerar um par de chaves pública e privada RSA quando o utilizador inicia execução, e efetua essa tarefa em caso positivo. Em caso negativo, pergunta pelo ficheiro onde está a chave privada;

Copyright © 2026 All rights reserved.

2. caso gere as chaves RSA, a chave pública fica guardada num ficheiro regular, enquanto que a chave privada fica guardada num ficheiro cifrado com AES-256-CBC, cuja chave de cifra é uma derivação segura de uma palavra-passe pedida ao utilizador;
3. a aplicação permite escrever um texto a ser assinado com uma notação que permite os cenários pouco usuais descritos na introdução deste tema;
4. a aplicação permite assinar um documento com as caracteristicas referidas em cima, produzindo o ficheiro com o texto e uma ou mais assinaturas digitais (note que, se o texto tiver alternativas, devem ser adicionadas assinaturas digitais para cada alternativa);
5. a aplicação permite validar assinaturas digitais, se lhe forem fornecidas a respetiva chave pública, o documento assinado e a assinatura digital.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. para casos de documentos com `[PLACEHOLDERs]` de escolha múltipla, a aplicação implementa um esquema eficiente em que inclui uma assinatura do *hash*(do ficheiro com o `[PLACEHOLDER]` + a opção respetiva);
2. a aplicação permite adicionar HMAC-SHA256 ao invés de assinaturas digitais;
3. a aplicação deve permitir escolher a cifra que é utilizada entre AES-256-CBC e AES-256-CTR para cifra do ficheiro da chave privada;
4. a aplicação usa também códigos de autenticação de mensagens como algoritmo de derivação segura de chaves de cifra a partir da palavra-passe (e.g., HMAC-SHA256);
5. a aplicação permite escolher a função de *hash* usada para produzir as assinaturas, e outros parâmetros relacionados com a segurança (e.g., o tamanho do módulo das chaves RSA);
6. outras funcionalidades que considere interessantes.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.4 EU-PAGOTAMANHÃ: Contratos e Empréstimos Seguros

EU-PAGOTAMANHÃ: Secure Contracts and Loans

O objetivo deste trabalho é desenvolver uma espécie de livro razão público (com possibilidade de ser pacialmente privado) onde se colocam *contratos* entre utilizadores registados do sistema. Este livro razão deve chamar-se `EU-PAGOTAMANHÃ`. Para obter uma ideia do que é pretendido, considere o seguinte cenário de utilização hipotético:

- Dois utilizadores registam-se na plataforma (apenas com nome, e-mail e palavra-passe), sendo-lhes automaticamente gerado um par de chaves pública e privada (e.g., RSA ou ElGamal). A chave privada é guardada pelo sistema, cifrada com uma derivação da palavra-passe do utilizador;
- Um dos utilizadores pede dinheiro emprestado ao outro, pelo que os dois resolvem fazer um *contrato*. O *contrato* é composto por texto, uma data, e duas assinaturas (uma de cada utilizador). Neste cenário, o texto pode ser algo semelhante a —
  `Devo-te 10 euros mas juro que te pago amanhã!`
  Contudo, o contrato pode ser o que os dois utilizadores muito bem entenderem;

Copyright © 2026 All rights reserved.

- Os dois utilizadores assinam o contrato. Um a referir que aceita, o outro a comprometer-se;
- Depois de assinado, o contrato é publicado para qualquer utilizador (registado ou não) na plataforma ver. Adicionalmente, qualquer utilizador (registado ou não) pode verificar a validade das assinaturas digitais;
- Opcionalmente, os utilizadores podem decidir que o contrato fica cifrado durante algum tempo, apesar do criptograma ficar sempre visível para todos;
- Caso o contrato termine, o utilizador que aceitou o contrato pode lá ir retirá-lo ou registá-lo como estando *sanado*.

Note-se que a ideia deste projeto é, no fundo, contruir uma rede social com um âmbito muito delimitado. A rede social permite fazer contratos (que são texto) seguros (assinados digitalmente) e publicá-los, para que o mundo seja testemunha. O sistema só terá dois atores diferentes (`utilizador registado` e `utilizador não registado`) e o conjunto de funcionalidades básicas a suportar pelo sistema são:

1. o sistema permite o registo de utilizadores (apenas com nome, e-mail e palavra-passe);
2. o sistema gera um par de chaves pública e privada RSA aquando do registo;
3. a chave pública fica imediatamente pública para todos os utilizadores (registados ou não) no perfil do utilizador registado; e
4. a chave privada fica guardada no sistema, idealmente cifrada (AES-256-CBC) com uma derivação da palavra-passe;
5. o sistema permite que dois utilizadores façam um contrato, escrevendo um texto, e que o assinem digitalmente (são duas assinaturas);
6. o sistema disponibiliza os contratos para qualquer utilizador, e permite a verificação das assinaturas (dentro do próprio sistema **e** disponibilizando os ficheiros, e.g., para que possa ser verificado com o `openssl`).

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. o sistema deve permitir que um contrato fique escondido (cifrado) durante algum tempo (o período temporal deve ser, nesse caso, pedido aos utilizadores);
2. o sistema deve permitir escolher a cifra que é utilizada entre AES-256-CBC e AES-256-CTR;
3. o sistema usa também códigos de autenticação de mensagens como algoritmo de derivação segura de chaves de cifra a partir da palavra-passe (e.g., HMAC-SHA256);
4. no caso do ponto anterior ser implementado, o sistema deve permitir escolher a função de HMAC usada entre HMAC-SHA256 e HMAC-SHA512;
5. outras funcionalidades que considere interessantes.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

Copyright © 2026 All rights reserved.

2.5 SEE-U-L4TER: Uma Cápsula do Tempo Criptográfica

SEE-U-L4TER: A Criptographic Time Capsule

O objetivo deste trabalho é desenvolver um sistema que permita cifrar mensagens (ficheiros) que só podem ser decifradas em determinado dia e hora. Para que seja seguro, o sistema deve implementar uma arquitetura cliente servidor (e.g., uma aplicação web), em que o o componente do servidor está claramente separado do cliente, e que tem como principais funcionalidades a geração de chaves de cifra para qualquer momento especificado e a cifra de mensagens com essas chaves. O sistema deve ter trs grandes funcionalidades, eventualmente disponibilizadas em três interfaces:

- usando o sistema, um utilizador deve poder pedir que lhe seja, por exemplo, cifrado um ficheiro que só pode ser aberto no dia e hora `25 de maio de 2030, 12h00`, introduzindo a data e o ficheiro através de alguma interface (e.g., um formulário). O sistema deve então gerar uma chave de cifra única AES que dependa de `um segredo` que só o sistema sabe, do `nome de utilizador` (e-mail) do utilizador que submete e da `data e hora`, cifrar o ficheiro e devolver o criptograma ao utilizador (sem a chave de cifra), para que o utilizador dele faça o que quiser, inclusive enviá-lo a outro utilizador;
- qualquer outro utilizador deve poder aceder ao sistema e verificar qual a chave que abre os critptogramas da data e hora **atual** numa interface não autenticada. Por outras palavras, qualquer utilizador tem acesso a chaves de decifra a todo o momento, mas cada chave só é mostrada nesse período de tempo, e nunca antes ou depois;
- adicionalmente, o sistema pode ter também uma interface que permite a submissão de uma chave de cifra e de um ficheiro cifrado anteriormente, devolvendo em resultado o ficheiro decifrado (se a decifra for bem sucedida).

Assim, o conjunto de funcionalidades básicas a suportar pelo sistema são:

1. o sistema gera chaves de cifra pseudo aleatoriamente usando um algoritmo criptográfico, mostrando-as numa interface isolada para a data atual na página de *landing*. As chaves de cifra são dependentes de um e-mail, de um segredo (que só o sistema sabe), e do dia e hora atuais. Por exemplo SHA256(e-mail || segredo || data e hora);
2. a pedido de um utilizador, o sistema gera chaves de cifra para um determinado dia e hora (usando a mesma técnica referida no ponto anterior), e cifra um ficheiro (submetido pelo utilizador) com essa chave;
3. aquando da operação no ponto anterior, o sistema gera também um código de autenticação de mensagem HMAC-SHA256, que é acoplado ao criptograma;
4. a pedido de um utilizador, o sistema tenta a decifra de um criptograma que lhe seja submetido usando a chave que o utilizador também fornecer;
5. aquando da execução da tarefa no ponto anterior, o sistema verifica o código de autenticação de mensagem e avisa o utilizador o código não verificar.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. o sistema deve permitir escolher a cifra que é utilizada entre AES-128-CBC e AES-128-CTR;
2. o sistema deve permitir escolher a função de HMAC usada entre HMAC-SHA256 e HMAC-SHA512;
3. para além de usar códigos de autenticação de mensagens para verificar se uma mensagem foi bem decifrada, o sistema usa assinaturas digitais RSA (neste caso, o sistema tem forma de gerar um par de chaves RSA ou tem-nas instaladas pelo administrador do sistema);

Copyright © 2026 All rights reserved.

4. o sistema permite registo simples de um utilizador através de um *e-mail* e palavra-passe (deve ser guardada uma representação segura da palavra-passe na base de dados);

5. caso a funcionalidade anterior seja implementada, o sistema deve permitir a utilizadores registados acederem a qualquer chave de cifra do passado (mas não as futuras);

6. outras funcionalidades que considere interessantes.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.6 CANTTOUCHME: Um Livro de Registos Pessoal e Seguro

CANTTOUCHME: A Personal and Safe Book of Records

A ideia deste trabalho é desenvolver um programa que permita guardar registos de texto imutáveis para vários utilizadores. Embora as aplicações ou sistemas que suportam mais do que um utilizador serem especialmente beneficiadas em arquiteturas cliente/servidor, esta pode ser pensada como uma aplicação para computador pessoal (quem quiser, pode fazer uma aplicação *web*), que eventualmente é usado por mais do que um utilizador. A aplicação deve permitir registar os utilizadores e que estes registem atividades do seu dia através da introdução de uma descrição em texto. Sempre que um utilizador entra na aplicação, este deve apenas ver um campo onde pode escrever essa descrição, que fica guardada juntamente com a data, e ter a opção de ver registos anteriores. A aplicação deve guardar todos os registos cifrados no sistema (para que ninguém os consiga ver, mesmo que abra os ficheiros com outro programa externo). A chave de cifra deve ser única por utilizador, e ser derivada da palavra-passe do utilizador. Embora se apresente como um aspeto mais avançado, recomenda-se que a aplicação guarde cada um dos registos com um código de autenticação de mensagens que o cole ao bloco anterior, ao estilo de uma *blockchain* (i.e., cada novo registo é adicionado como sendo um novo bloco da *blockchain*). Assim, cada novo bloco contém
`[valor hash do bloco anterior, data do registo, texto do registo, código de auteticação de mensagem]`.
A chave de integridade deve ser também única por utilizador, e portanto ser derivada da sua palavra-passe.

Assim, o conjunto de funcionalidades básicas a suportar pela aplicação (e que lhe definem também um pouco melhor o comportamento) são:

1. a aplicação permite que o utilizador se registe com um *e-mail* e palavra-passe, devendo ser guardada uma representação segura da palavra-passe na base de dados ou ficheiro de utilizadores;

2. a aplicação deve gerar automaticamente uma chave para cifra/decifra dos registos a partir da palavra-passe do utilizador e de um número aleatório associado a esse utilizador;

3. a aplicação deve gerar automaticamente uma chave para integridade (código de autenticação de mensagens) dos registos a partir da palavra-passe do utilizador e de um outro número aleatório associado a esse utilizador;

4. a aplicação deve permitir que um utilizador insira registos e cifrar todos os registos com a chve de cifra/decifra mencionada em cima utilizando AES-128-CBC;

5. a aplicação deve guardar códigos de autenticação de mensagens para cada registo feito pelo utilizador. A chave de integridade a usar é a que foi mencionada em cima;

Copyright © 2026 All rights reserved.

6. a aplicação deve permitir que um utilizador veja os seus registos anteriores. Portanto, quando um utilizador entra na aplicação, esta deve permitir decifrar e mostrar quaisquer registos anteriores.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. a aplicação verifica o código de autenticação de mensagens antes de mostrar registos anteriores ao utilizador, e mostra o resultado da validação (`válido` ou `não válido`) junto com o registo;
2. a aplicação deve permitir escolher a cifra que é utilizada entre AES-128-CBC e AES-128-CTR, aquando do registo do utilizador;
3. a aplicação deve permitir escolher a função de HMAC usada entre HMAC-SHA256 e HMAC-SHA512, aquando do registo do utilizador;
4. para além de usar códigos de autenticação de mensagens para verificar se um bloco/registo foi bem decifrado, o sistema usa assinaturas digitais RSA (neste caso, o sistema tem forma de gerar um par de chaves RSA de sistema);
5. a aplicação implementa o sistema de *blockchain* mencionado na introdução.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação.

2.7 NOTHING2LOSE: Uma Lotaria que Compromete Tempo e Prémios

NOTHING2LOSE: A Lottery that Compromises Time and Prizes

A criptografia contém ferramentas excelentes para desenvolver jogos de mistério com desafios. A ideia deste trabalho é desenvolver um sistema que gere quatro bilhetes de lotaria com prémios cifrados e permita que os seus utilizadores decifrem um desses bilhetes. Os quatro bilhetes devem ter valores diferentes (e.g., prémio simples de 1 runa; prémio médio de 2 runas; prémio raro de 4 runas e prémio lendário de 8 runas), e serem incrementalmente dificeis de abrir. Por outras palavras, se um utilizador escolher um bilhete que tem um prémio maior (sem saber), este prémio deve demorar mais tempo a decifrar (por exemplo, o bilhete com o prémio simples deve demorar 30 segundos, enquanto o prémio médio deve demorar o dobro, i.e., 60 segundos; o prémio lendário deve demorar 4 minutos). O sistema a construir deve, portanto, simular tudo isto:

1. O sistema gera quatro bilhetes com diferentes prémios (os prémios devem ser aleatórios, mas com valores definidos por uma exponencial);
2. O sistema gera quatro chaves de cifra com complexidade diferente (e.g., a chave para o prémio simples tem 20 bits aleatórios + 108 bits preenchidos com 0s; a chave para o prémio médio tem 21 bits aleatórios + 107 bits preenchidos com 0s; a chave para o prémio raro tem 22 bits aleatórios + 106 bits preenchidos com 0s; e a chave para o prémio lendário tem 23 bits aleatórios + 105 bits preenchidos com 0s);
3. O sistema calcula códigos de autenticação de mensagens para os vários prémios;
4. O sistema cifra os quatro bilhetes;
5. O sistema pede ao utilizador que escolha um prémio e inicia a decifra desse prémio até devolver o resultado ao utilizador (o utilizador tem de esperar até receber o prémio ou desistir, e escolher outro).

Assim, o conjunto de funcionalidades básicas a suportar pelo sistema são:

Copyright © 2026 All rights reserved.

1. registo simples de um utilizador através de um *e-mail* e palavra-passe (deve ser guardada uma representação segura da palavra-passe na base de dados);
2. o sistema gera bilhetes com diferentes prémios seguindo o esquema em cima;
3. o sistema gera chaves de cifra com diferentes complexidades (testem as complexidades para saberem quantos bits devem ter para demorarem 30, 60, 120 e 240 segundos a decifrar);
4. o sistema calcula códigos de autenticação de mensagem com HMAC-SHA256;
5. o sistema cifra os bilhetes com as chaves geradas usando AES-128-CBC;
6. o sistema permite a um utilizador registado escolher um bilhete e tenta decifrá-lo (demorando o tempo necessário), e mostrando o resultado no final;
7. o sistema permite que o utilizador desista da decifra de um bilhete (se estiver demorar muito), e que escolha outro, até que não tenham mais nenhum para escolher.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. o sistema deve permitir escolher a cifra que é utilizada entre AES-128-CBC e AES-128-CTR;
2. o sistema deve permitir escolher a função de HMAC usada entre HMAC-SHA256 e HMAC-SHA512;
3. para além de usar códigos de autenticação de mensagens para verificar se uma mensagem foi bem decifrada, o sistema usa assinaturas digitais RSA (neste caso, o sistema tem forma de gerar um par de chaves RSA de sistema);
4. Inventar esquemas de ajuda inteligentes para decifra (e.g., responder corretamente a uma pergunta permite reduzir o tempo de decifra por desvendar parte da chave de cifra).

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.8 CHALLENGE-ACCEPTED: Um Sistema de Publicação de Desafios Criptográficos

CHALLENGE-ACCEPTED: System to Publish Cryptographic Challenges

A criptografia contém ferramentas excelentes para desenvolver jogos de mistério com desafios. A ideia deste trabalho é desenvolver um sistema que permita que diferentes utilizadores publiquem e resolvam desafios. Os desafios podem ser mensagens cifradas com diferentes cifras ou encontrar determinados valores de *hash*. O sistema deve permitir que os utilizadores se registem (todos os utilizadores são iguais) e deixem desafios criptográficos para que outros utilizadores registados os possam resolver. Só deve existir um tipo de utilizador (i.e., um Utilizador pode simultaneamente publicar ou resolver desafios). Devem existir dois tipos básicos de desafios (embora a equipa que desenvolve o trabalho possa definir mais):

1. Desafios de decifra de mensagens, em que é preciso adivinhar uma palavra-passe para decifrar um ficheiro;

Copyright © 2026 All rights reserved.

2. Desafios de descoberta de mensagens para os quais são conhecidos os valores de *hash*.

Assim, o conjunto de funcionalidades básicas a suportar pelo sistema são:

1. registo simples de um utilizador através de um *e-mail* e palavra-passe (deve ser guardada uma representação segura da palavra-passe na base de dados);
2. cada utilizador deve poder colocar no sistema desafios do tipo cifra de mensagens. Neste caso, para além da mensagem, o sistema deve pedir uma palavra-passe, derivar uma chave de cifra, e cifrar essa mensagem, codificando-a em BASE64;
3. para que a funcionalidade anterior resulte, o sistema deve também calcular um código de autenticação de mensagens (com uma chave secreta colocada pelo administrador do sistema) que permite verificar se uma mensagem foi bem decifrada ou não, quando uma solução é submetida;
4. cada utilizador deve poder colocar no sistema desafios do tipo valor de *hash*. Neste caso, o sistema deve pedir apenas a mensagem, calcular o valor de *hash* e publicar esse valor em hexadecimal ou BASE64;
5. cada utilizador deve poder escolher um desafio (diferente dos que ali tiver colocado) e submeter uma solução a esse desafio. O sistema deve dizer se a solução submetida está correta ou não. No caso das mensagens cifradas, deve também mostrar a mensagem;
6. o sistema deve permitir fazer desafios com as cifras AES-128-ECB, AES-128-CBC e AES-128-CTR;
7. o sistema deve permitir fazer desafios com as funções de *hash* MD5, SHA256 e SHA512.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

1. o sistema não deve permitir mais do que uma tentativa a cada 15 segundos para os desafios cifrados (i.e., o processo de derivação de chaves de cifra a partir da palavra-passe deve ser propositadamente lento, para que a decifra seja propositadamente lenta também – ver projeto anterior);
2. para além de usar códigos de autenticação de mensagens para verificar se uma mensagem foi bem decifrada, o sistema usa assinaturas digitais RSA (neste caso, o sistema tem forma de gerar um par de chaves RSA para o administrador do sistema);
3. para além de suportar as cifras referidas nos pontos básicos, o sistema suporta ainda a ElGamal;
4. o sistema contém outros tipos de desafios criptográficos.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.9 MON-AMOUR: Um Sistema para Trocar Cartas de Amor, mas com um Senão...

MON-AMOUR: System to Exchange Love Letters, with a Twist...

Algumas cifras simples ou formas de esconder mensagens ou significados estão ligadas a episódios de trocas de segredos românticos. A ideia deste trabalho é fazer um sistema

Copyright © 2026 All rights reserved.

para cifrar e decifrar mensagens, eventualmente com a índole referida, mas com um pequeno senão: a cifra de uma mensagem deve ser feita a partir de uma palavra-passe (e.g., *"azul"*) e esta deve garantir que as tentativas de decifra do respetivo criptograma demoram sempre, no mínimo, 15 segundos (para criar *suspense*). Para se conseguir o efeito desejado, o sistema deve ser constituído por dois programas (um para cifra, outro para decifra). O programa para cifrar deve:

1. pedir ao utilizador para colocar uma pergunta para a qual o destinatário da mensagem deve conhecer a resposta (e.g., *"Qual a minha cor favorita?"*);
2. pedir ao utilizador para escrever a resposta à pergunta (e.g., *"azul"*), colocando todas as letras da resposta em minúsculas;
3. pedir ao utilizador para escrever a mensagem;
4. gerar um número aleatório n com 128 bits;
5. juntar a palavra-passe e o número aleatório, e calcular repetidamente o valor de *Secure Hash Algorithm 256* (SHA256) dessa junção e valores de *hash*, parando quando o tempo de execução for de 15 segundos ou mais, guardando o resultado final e o número de vezes que calculou o SHA256.

No final, o programa para cifrar deve cifrar a mensagem com a *Advanced Encryption Standard* (AES), num modo conveniente e usando como chave o valor de *hash* resultante do último passo. Finalmente, deve devolver um ficheiro com a seguinte estrutura:

`número de iterações` *`hash`* `| Pergunta | Número aleatório | Criptograma` .

Por sua vez, programa para decifrar deve:

1. Pedir o nome do ficheiro onde está o criptograma (ou este deve ser-lhe dado como parâmetro);
2. Mostrar a pergunta ao utilizador e pedir a resposta;
3. Tentar decifrar o criptograma, voltando a calcular o valor de *hash* iterativamente com os parâmetros fornecidos (i.e., número de iterações necessárias, palavra-passe e número aleatório).

Notem que o *senão* é o facto da decifra demorar algum tempo e ser inerentemente resistente a ataques de dicionário.

Assim, o conjunto de funcionalidades básicas a suportar pelo sistema são:

- permitir cifrar uma mensagem seguindo o esquema referido antes;
- permitir decifrar o criptograma após correta inserção da resposta à pergunta;
- inclusão de um *Hash based Message Authentication Code* (HMAC) (para além da estrutura acima referida) aquando da cifra de uma mensagem;
- verificação automática da integridade da decifra através do HMAC.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

- o programa de cifra permite a um utilizador gerar ou importar um par de chaves *Rivest, Shamir e Adleman* (RSA);
- para além do HMAC, é calculada e incluída uma assinatura digital aquando da cifra de uma mensagem (neste caso, a chave pública de verificação também pode ser incluída na estrutura do ficheiro produzido);

Copyright © 2026 All rights reserved.

- aquando de uma decifra bem sucedida, é também verificada a assinatura digital da mensagem;
- as chaves privadas RSA são geradas e guardadas de forma segura imediatamente (cifradas com uma cifra de chave simétrica);
- considerar desenvolver um esquema de confiança para as chaves públicas, eventualmente através do desenvolvimento de um programa que simule um agente de confiança que gera chaves RSA e as certifica;
- escrever um *help* bastante completo.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.10 CTRL+C: Um Gestor de *Clipboard* Porreirinho

CTRL+C: An Extremely Nice Clipboard Manager

O objetivo principal deste projeto é desenvolver um gestor do *Clipboard* com alguma segurança embutida. Estes gestores são programas que permitem guardar todo o histórico da utilização do *Clipboard*, e até aceder às entradas anteriores com bastante facilidade, sendo esta uma funcionalidade de grande utilidade e uma das principais motivações para a sua existência. O problema é que alguns (senão todos os) gestores de *Clipboard* guardam o histórico num ficheiro de texto em texto-limpo, inclusive palavras-passe que possam estar a ser copiadas de um gestor de palavras-passe para um formulário *online*, ficando assim expostas a cavalos de Tróia ou até a utilizadores concorrentes do sistema. Pretende-se que o programa desenvolvido no contexto deste projeto resolva esse problema e adicione valor através do suporte das seguintes funcionalidades:

- o histórico do *Clipboard* deve ser guardado cifrado com uma cifra (e modo de cifra) de qualidade reconhecida (e.g., *Advanced Encryption Standard* (AES));
- a chave de cifra deve ser gerada aquando da execução do gestor e guardada apenas em memória. A geração da chave de cifra deve ser feita de forma segura;
- o histórico deve ser salvo no disco (cifrado) a cada 5 minutos (apagando o anterior);
- devem ser criados valores de *hash* de cada entrada no histórico, que devem ser guardados em texto limpo num ficheiro à parte;
- deve ser possível verificar se determinada entrada existe no último ficheiro do histórico cifrado, sem o decifrar, e recorrendo apenas aos valores de *hash*;
- deve ser feita uma assinatura digital do ficheiro cifrado aquando este é guardado no disco (i.e., a cada 5 minutos). Para tal, deve ser gerado um par de chaves *Rivest, Shamir e Adleman* (RSA) que acompanha a aplicação.

Este projeto pressupõe o desenvolvimento de uma aplicação que procura obter as entradas do *Clipboard* em segundo plano. Para tal, pode pensá-la como uma aplicação *Client Line Interface* (CLI) com uma *thread* que core em corre em segundo plano e que simula um `CTRL+v` a cada 2 segundos, na expetativa de encontrar uma entrada nova. Eventualmente, pode ainda desenvolver uma aplicação para configuração de alguns parâmetros (e.g., quantas entradas são suportadas). A aplicação principal deve permitir ver as várias entradas no *Clipboard* e também escolher qual deve ir para lá através de duplo clique. Note que a versão mais simples da aplicação, o histórico é perdido sempre que esta é terminada (porque a chave de cifra que permite decifrar o ficheiro está apenas guardada em memória).

Copyright © 2026 All rights reserved.

Uma versão mais elaborada da aplicação deve ter funcionalidades adicionais, como as que se sugerem a seguir:

- o gestor deve suportar a geração de chaves de cifra a partir de uma pitada de sal e de uma palavra-passe mestra, configurada pelo utilizador aquando da primeira utilização. Deve ser usada, para o efeito, uma função de derivação de chaves de cifra como a PBKDF2;
- tomando a funcionalidade anterior como garantida, o gestor deve conseguir decifrar ficheiros de históricos anteriores;
- tomando a funcionalidade anterior como garantida, o gestor deve conseguir decifrar, de forma muito seletiva, apenas as entradas selecionadas pelo utilizador. Esta funcionalidade requer que seja usado um modo de cifra adequado;
- o gestor deve suportar vários utilizadores (cada utilizador terá o seu histórico);
- as assinaturas devem ser verificadas a pedido do utilizador;
- permitir que os programas cliente utilizem certificados digitais para validação das assinaturas digitais;
- escrever um *help* bastante completo.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação.

2.11 TWO-HEADED-BEAST: Aplicação para Transmitir Ficheiros entre Dois Utilizadores

TWO-HEADED-BEAST: Application to Transmit Files Between Two Users

O objectivo primordial deste projeto é o de construir uma aplicação que permita transmitir ficheiros potencialmente grandes entre dois utilizadores, de um modo seguro. Entre outras, encontram-se as seguintes funcionalidades básicas da solução:

- para ficheiros maiores que 12Mb, o programa deve repartir o ficheiro e operar sobre estes blocos; é claro que é da responsabilidade do programa garantir que a integridade de *cada* bloco é assegurada;
- caso o utilizador escolha não cifrar o ficheiro a transmitir, este deve ser acompanhado de um código de autenticação da mensagem;
- as chaves de sessão devem ser trocadas utilizando o protocolo de acordo de chaves Diffie-Hellman ou através de criptografia de chave pública (pode recorrer ao `OpenSSL` para esta funcionalidade específica);
- o utilizador deve poder optar por comprimir o ficheiro antes de o cifrar.

Note que, a definição deste projeto presume que dois programas são colocados a correr em ambos os lados da comunicação, e que um deles vai ficar à espera que o outro se ligue a ele. Logo que se liguem, acorda-se a chave de sessão, o emissor cifra o ficheiro e envia para o segundo. É da responsabilidade do emissor a partição do ficheiro em blocos (se aplicável), a compressão, a cifragem e a aplicação dos mecanismos de integridade, garantia de autenticação da mensagem ou assinatura digital (ver abaixo). É da responsabilidade do receptor a decifragem, decompressão e montagem do ficheiro recebido, bem como a verificação de que o ficheiro está integro e veio de fonte segura. Caso algo corra mal (por exemplo, o ficheiro não passa na verificação de integridade), deve ser exibido um erro em ambos os lados da comunicação.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

Copyright © 2026 All rights reserved.

- possibilitar a anexação da assinatura digital ao ficheiro através da especificação do ficheiro com a chave privada e recorrendo, como opção, ao `OpenSSL`.
- permitir validar a assinatura digital de um ficheiro recorrendo a uma chave pública (supostamente de quem assinou) ou de um certificado digital (podem recorrer ao `OpenSSL` para esta tarefa).
- permitir que o utilizador escolha a cifra a utilizar e o comprimento da chave de cifra;
- permitir que o utilizador escolha a função de *hash* a usar;
- ter um *help* bastante completo.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação.

Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.12 iCHATO: Aplicação para Conversação Online com Mensagens Assinadas

iCHATO: Application for Online Chatting with Signed Messages

Nota: este tema só está disponível para Engenharia Informática.

Com este projeto pretende-se que o grupo de trabalho desenvolva uma aplicação para conversasão *online* (vulgarmente conhecida como *chat*) tipo *Internet Relay Chat* (IRC), mas cujas mensagens são assinadas e, opcionalmente, cifradas. A plataforma desenvolvida deve suportar as seguintes funcionalidades:

- registo de novos utilizadores (neste caso, a plataforma deve gerar um par de chaves pública e privada automaticamente e atribui-las ao utilizador de modo seguro);
- permitir que os utilizadores falem todos uns com os outros num canal público (neste caso, as mensagens só seguem assinadas, não cifradas);
- permitir que dois utilizadores falem um com o outro diretamente através de um canal dedicado, com mensagens cifradas e assinadas (as chaves de sessão podem ser trocadas usando criptografia assimétrica ou o protocolo de acordo de chaves Diffie-Hellman).

Notem que este projeto pressupõe o desenvolvimento de duas aplicações diferentes: uma que atua como cliente; e outra que atua como servidor. A aplicação cliente é a que serve de interface para o sistema, e a que permite que um utilizador se registe da primeira vez que se liga à plataforma. É o servidor que guia o processo de registo, guardando o `nome de utilizador`, a `palavra-chave` e algumas informações do utilizador. É também o servidor que gera e envia o par de chaves pública/privada (e o certificado, se aplicável - ver funcionalidade extra em baixo) para o cliente. Notem que devem tomar as devidas precauções para que o referido envio seja feito de modo seguro (i.e. trocando chaves de sessão e cifrando as chaves ou certificados durante este envio). As chaves públicas (ou certificados) podem ser guardadas no servidor e distribuídas a quem as requisitar durante o funcionamento da aplicação, mas as chaves privadas devem ficar somente no lado do cliente a quem pertencem.

O utilizador registado acede ao sistema via introdução do seu `nome de utilizador` e da `palavra-passe`, e após validado, passa a poder escrever no canal público do sistema. Caso deseje falar com alguém em particular, deve ser gerada e trocada uma chave de sessão, usada para garantir a confidencialidade do canal assim criado.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

Copyright © 2026 All rights reserved.

- guardar as palavras-chave de uma forma segura;
- permitir que mais do que dois utilizadores falem de modo privado com mensagens cifradas;
- permitir que os programas cliente utilizem certificados digitais para validação das assinaturas digitais;
- a implementação da funcionalidade anterior implica que a aplicação servidor deve adicionalmente facilitar a criação do certificado digital aquando da geração do par de chaves pública/privada que ocorre durante o registo de um utilizador.
- escrever um *help* bastante completo.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

2.13 MR.SMITH: Plataforma para Cifrar e Assinar usando um Agente de Confiança

MR.SMITH: Platform for Encrypting and Digitally Signing Files using a Trusted Third Party

O objetivo principal deste trabalho é o de construir uma plataforma para assinar e transmitir ficheiros de um modo seguro, entre dois ou mais utilizadores, recorrendo apenas a mecanismos da criptografia simétrica e a um agente de confiança. Entre outras, apontam-se as seguintes funcionalidades básicas do sistema a desenvolver:

- deve ser possível fazer o registo junto do agente de confiança;
- o utilizador, depois de devidamente autenticado no sistema, deve poder escolher se quer cifrar ou assinar um ficheiro antes de o enviar; deve também poder escolher o utilizador para o qual quer enviar o ficheiro a partir de uma lista ligados ao sistema;
- caso o utilizador escolha não cifrar o ficheiro a transmitir, este deve ser acompanhado de um código de autenticação da mensagem (deve ser usado um *Hash based Message Authentication Code* (HMAC) para o efeito);
- as chaves de sessão devem ser geradas por um dos clientes e trocadas via agente de confiança;
- o agente de confiança deve ser capaz de fazer assinaturas digitais de ficheiros transmitidos por utilizadores autenticados do sistema.

Note que a definição deste projeto indicia que o sistema só poderá estar completamente operacional quando estão ligados, pelo menos, três intervenientes: (i) a aplicação (cliente) que vai ser usada para transmitir o ficheiro, (ii) a aplicação (cliente) que vai ser usada para receber o ficheiro e (iii), o agente de confiança. Pressupõe-se que o agente de confiança já está a correr quando qualquer utilizador se tenta ligar ao sistema, que é este que gera todas as chaves de sessão para confidencialidade nas comunicações, e que alimenta cada cliente com as informações acerca de outros clientes ligados ao sistema (para que determinado cliente possa enviar um ficheiro a outro, este deve estar *online*). Pressupõe-se que as chaves de cifra de chaves de sessão são inseridas manualmente no sistema aquando da inicialização de determinado cliente.

Podem fortalecer o trabalho e conhecimento através da implementação das seguintes funcionalidades:

- permitir que apenas as primeiras chaves de cifra de chaves de sessão sejam trocadas usando criptografia de chave pública.

Copyright © 2026 All rights reserved.

- garantir que as credenciais de autenticação no servidor / agente de confiança (i.e. *username* e *password*) são trocadas de forma segura.
- assegurar que as chaves de cifra de chaves de sessão mudam sempre que o utilizador faz *login* no sistema, sem que se perca sincronismo e nunca recorrendo a mecanismos de criptografia simétrica (pesquisar por *one time password*);
- permitir que o utilizador escolha a cifra a utilizar e o comprimento da chave de cifra;
- ter um *help* bastante completo.

Pensem numa forma de atacar o sistema (uma falha da sua implementação) e dediquem-lhe um pequeno intervalo de tempo na apresentação. Notem que, para efeitos de avaliação e prototipagem, o sistema desenvolvido pode executar localmente todos os seus componentes/aplicações/programas, desde que simule ou concretize a arquitetura sugerida (i.e., não precisa necessariamente executar em rede).

Copyright © 2026 All rights reserved.